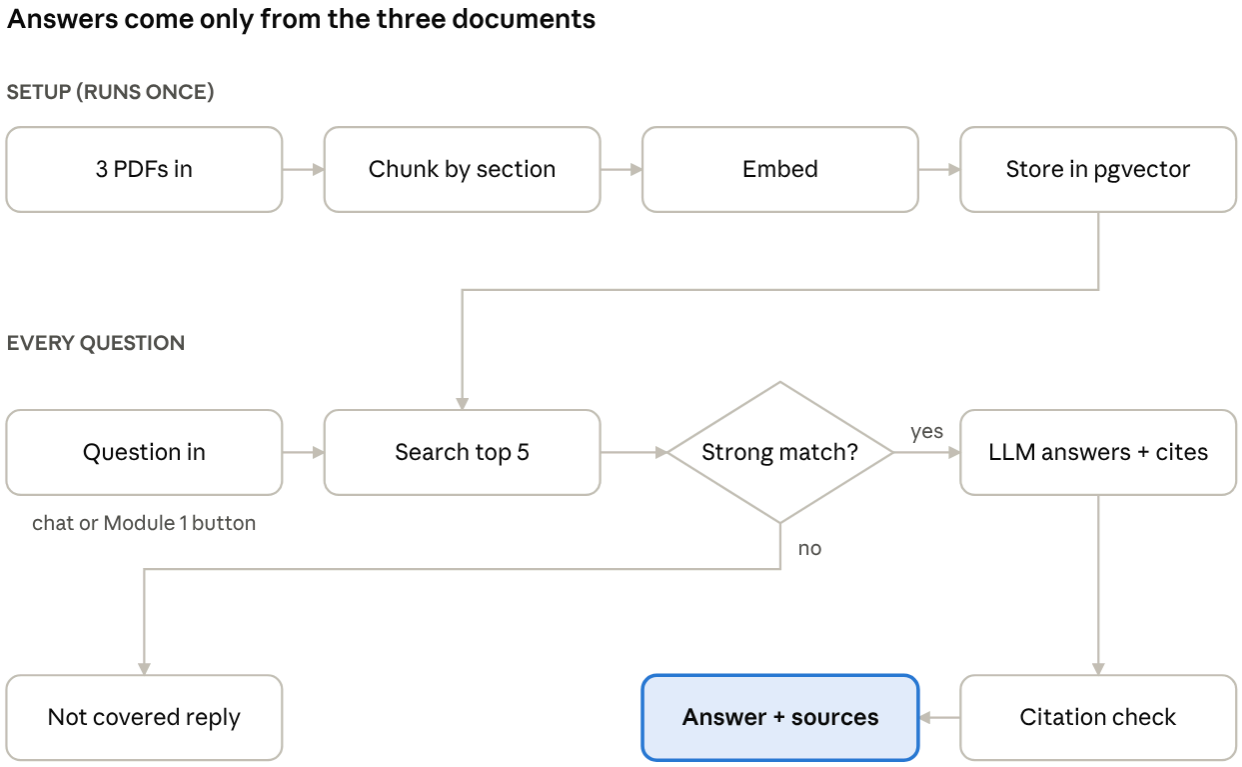
MediFlow AI — Module 2 Architecture (Protocol Assistant)

Module 2 is a RAG system that answers questions only from three clinic documents, always with citations, and checks each Module 1 prescription against the guidelines. Each section below can be handed to a coding agent as-is.

Flow at a glance



Module 2 flow · setup + question pipelines

Setup builds the searchable knowledge base once. Every question, from the chat panel or the Module 1 Check guideline button, is searched against it; a weak match is refused before the LLM is ever called.

Role in the product

Module 2 is the product's rulebook. Module 1 records what the doctor did; Module 2 checks it against what the clinic's guidelines say should be done.

Two ways it is used

1. **Ask the guidelines (chat panel).** Any staff member types a question, e.g. "When should a malaria patient be referred?", and gets an answer with document and page citations.
2. **Check guideline (button on the Module 1 review screen).** It takes the diagnosis and medicines from the draft note, finds the matching guideline section, and shows, per drug, whether the prescription matches. Differences are highlighted for the doctor. It is advice only; the doctor still decides and approves.

Connection to Module 1: Module 1 provides the input (diagnosis, diagnosis code, medications). Module 2 adds a guideline check on the review screen *before* the doctor clicks Approve. The result of the check is saved with the note.

Connection to later modules: the Care coordinator (Module 5) will call it to look up follow-up and referral rules. The Claims checker (Module 6) will use the drug list document to confirm NHIS coverage.

The knowledge base

Three synthetic PDFs, about 30 pages in total, stored in `data/knowledge_base/`. Every page carries the footer "Fictional document for a software prototype. Not for clinical use."

Document	File	Sections	Used later by
Treatment Guidelines	`treatment_guidelines.pdf`	One section per Module 1 diagnosis (10): definition, symptoms, tests, first-line treatment with doses, second-line treatment, danger signs, when to refer, follow-up interval	Module 1 check, Module 5
Hospital Procedures	`hospital_procedures.pdf`	Triage levels, referral process, discharge process, follow-up booking rules, patient education basics	Module 4, Module 5
Drug List	`drug_list.pdf`	One entry per drug: name, form, adult dose, child dose, max daily dose, NHIS covered (yes/no), NHIS tariff in GHS	Module 1 check, Module 6

Rule that ties it to Module 1: every medicine in the 10 Module 1 answer keys must appear in the Drug List and in the matching treatment section. The guidelines are written to match 8 of the 10 Module 1 prescriptions and to differ on 2 (a different dose, or the drug listed as second-line), so the Check guideline button has something to catch. Module 1's files are not changed.

Components

Two pipelines: **setup** runs once (and again whenever a document changes); **question** runs every time someone asks.

Setup pipeline

C1. Load documents

- **Input:** the 3 PDFs in `data/knowledge_base/`.
- **Job:** extract text page by page, keeping headings and page numbers (use `pymupdf`).
- **Output:** text with page numbers and the heading each paragraph sits under.

C2. Chunk

- **Job:** cut text into pieces by section heading. Target 200–500 words per chunk; if a section is longer, split it with a 50-word overlap.
- **Each chunk keeps:** document name, section title, page number(s), and a chunk ID like `TG-malaria-02`.
- **Why by section:** a guideline section is a complete idea; cutting mid-section separates a drug from its dose.

C3. Embed and store

- **Job:** turn each chunk into a vector (a list of numbers capturing its meaning) and save it.
- **Embedding model:** a free open-source model run locally (e.g. `BAAI/bge-small-en-v1.5` via sentence-transformers). It costs nothing and the text never leaves our system.
- **Store:** PostgreSQL with pgvector, plus a full-text index on the same chunk text for keyword search.

Question pipeline

C4. Receive question

- **From the chat panel:** the typed question.
- **From the Check guideline button:** the system builds the question from the Module 1 draft, e.g. "Recommended treatment for uncomplicated malaria (B54)" plus the list of prescribed drugs. It runs one search for the treatment section and one per drug in the Drug List, so dose and NHIS coverage are both retrieved.

C5. Search (hybrid)

- **Meaning search:** embed the question and find the 10 closest chunks by vector similarity.
- **Keyword search:** find the 10 best full-text matches. This catches exact drug names and codes that meaning search can miss.
- **Merge:** combine both lists with Reciprocal Rank Fusion (a simple formula that rewards chunks ranked high in either list) and keep the top 5.
- **Output:** 5 chunks with their scores and citations.

C6. Strength check

- If the best vector similarity is below the threshold (start at 0.50; tune it on the test set), stop and reply "This is not covered in the clinic guidelines." The LLM is not called.

C7. Generate answer

- **Job:** send the question and the 5 chunks to Claude with the answer prompt (or the Check guideline prompt). Temperature 0.

- **Output:** an answer where every statement carries a citation in the form `[TG-malaria-02]`.

C8. Citation check and display

- **Job:** confirm every cited chunk ID is one of the 5 retrieved. Then turn IDs into readable sources: "Treatment Guidelines, Malaria, p. 4".
- **Display:** answer on top, sources listed below, each source expandable to show the original chunk text.

The LLM prompts

Two prompts, stored as `prompts/protocol_answer_v1.txt` and `prompts/protocol_check_v1.txt`.

Answer prompt (chat panel)

```
You answer questions for staff at an outpatient clinic in Ghana, using
ONLY the guideline excerpts below.

Rules:
1. Use only the excerpts. Do not use outside medical knowledge.
2. Put the chunk ID in square brackets after every statement,
   e.g. [TG-malaria-02].
3. If the excerpts do not answer the question, reply exactly:
   "This is not covered in the clinic guidelines."
4. If the excerpts only partly answer it, answer that part and say
   what is not covered.
5. Keep answers under 150 words. Use short bullet points for doses.

Excerpts:
{chunks_with_ids}

Question: {question}
```

Check guideline prompt (Module 1 button)

Forced into JSON with the tool-use schema below, so the screen can colour each drug.

```
You compare a doctor's prescription with the clinic guideline excerpts
below. Use ONLY the excerpts.

For each prescribed drug, return:
- drug: name as prescribed
- status: one of "matches", "dose_differs", "not_first_line",
  "not_in_guideline"
- guideline_says: what the guideline recommends, in one line
- citation: the chunk ID supporting it
Also return:
- missing_from_prescription: first-line drugs the guideline lists
  that were not prescribed (empty list if none)
- nhis_covered: for each drug, yes/no/unknown from the Drug List

Diagnosis: {diagnosis} ({diagnosis_code})
Prescribed: {medications}

Excerpts:
{chunks_with_ids}
```

Guardrails

The rule behind every guardrail: if the documents don't say it, the assistant doesn't say it.

Layer	Rule	What happens
1. Weak match	Best vector similarity below the threshold (start 0.50)	Reply "This is not covered in the clinic guidelines." LLM not called
2. Prompt rule	Answer only from the excerpts; cite every statement	Model refuses or answers partly
3. Citation check	Every cited chunk ID must be one of the 5 retrieved	Answer shown with a warning banner "Some sources could not be verified"
4. No citations	An answer with zero citations that is not the refusal sentence	Answer blocked; refusal sentence shown instead
5. Schema check (Check guideline)	JSON matches the schema; status is one of the 4 allowed values	Retry once, then show "Guideline check unavailable"

Layer	Rule	What happens
6. API error or timeout (20 seconds)	Retry twice, waiting 2 then 4 seconds	Show "Assistant unavailable, try again"

Always on screen: "Answers come from the clinic's guideline documents. Clinical decisions remain with the doctor."

Storage, logging and tech

Module 2 adds two tables to the same database as Module 1, plus one column on the Module 1 notes table.

`kb_chunks` (the knowledge base)

Column	What it holds
chunk_id	e.g. `TG-malaria-02`
document, section, pages	Where it came from
text	The chunk text (with a full-text index)
embedding	The vector (pgvector)
doc_version	Changes when a PDF is re-ingested

`rag_queries` (log of every question)

Column	What it holds
id, source	Query ID; `chat` or `check_guideline`
question	As asked or as built from the note
retrieved_chunks, scores	The 5 chunk IDs and their scores
answer, citations	What was shown
refused, flags	Whether it refused; any guardrail flags
model, prompt_version, latency_ms, created_at	For measuring and debugging

On Module 1's `clinical_notes`: add `guideline_check` (the JSON result) and `guideline_check_query_id`, so every approved note shows whether it was checked and what was found.

Tech

Part	Choice
PDF reading	pymupdf
Embeddings	sentence-transformers, `BAAI/bge-small-en-v1.5`, run locally
Vector + keyword search	PostgreSQL with pgvector and built-in full-text search
LLM	Claude via the same provider layer as Module 1
Evaluation	Python script + Ragas (or an LLM-as-judge script)

API endpoints

- `POST /kb/ingest`: runs C1–C3 on everything in `data/knowledge_base/`.
- `POST /kb/ask` with a question: returns answer, citations, refused flag.
- `POST /notes/{id}/check-guideline`: runs the Check guideline flow for a Module 1 draft and saves the result on the note.
- `GET /kb/chunks/{chunk_id}`: returns the original text for a citation.

KPIs and evaluation

Module 2 is measured on a fixed test set of 30 questions (25 answerable, 5 deliberately outside the documents), plus the 10 Module 1 notes for the Check guideline flow. One command, `python evals/run_module2.py`, runs everything and writes a results table to `docs/evals.md`.

KPI	How it is measured	Target
Retrieval hit rate	The expected chunk is in the top 5, across the 25 answerable questions	90% or more
Answer accuracy	Answer agrees with the expected answer and adds nothing unsupported (LLM-as-judge, then you spot-check 5 by hand)	90% or more
Citation accuracy	Cited chunks actually contain the cited statement	90% or more
Refusal accuracy	The 5 out-of-scope questions get the refusal sentence	5 of 5

KPI	How it is measured	Target
False refusals	Answerable questions wrongly refused	1 or fewer
Check guideline catch rate	The 2 planted mismatches are flagged; the 8 correct ones are not	10 of 10 correct
Speed	Time from question to answer shown	Under 5 seconds

The refusal threshold in C6 is tuned with this test set: raise it if out-of-scope questions get answered, lower it if answerable ones get refused.

Definition of done

Module 2 is done when all of these work:

- [ ] The 3 PDFs ingest with one command, and chunk count and IDs are printed
- [ ] The chat panel answers a guideline question with readable citations, and clicking a citation shows the original text
- [ ] An out-of-scope question (e.g. "How do I treat a broken leg?") gets the refusal sentence
- [ ] The Check guideline button on the Module 1 review screen colours each drug by status
- [ ] The 2 planted mismatches are flagged and the result is saved on the note
- [ ] Every query appears in `rag_queries` with chunks, scores and latency
- [ ] The evaluation script runs and every KPI meets its target, or the gap is written up in `docs/evals.md`

Synthetic data prompts

Run Prompt A after the Module 1 transcripts exist, because the documents are built to match them. Run Prompt B after Prompt A.

Prompt A: the three PDFs

```
Create three fictional clinic documents for a software prototype and
save them as PDFs in data/knowledge_base/. Write the content first as
Markdown (keep the .md files too), then convert to PDF with clear
headings and page numbers. Every page footer must read:
"Fictional document for a software prototype. Not for clinical use."
The clinic is "MediFlow Clinic", a fictional outpatient clinic in Ghana.

First, read the answer_key in every file in data/transcripts/ and list
```

```
every diagnosis and every medication (name, dose, frequency, days).

1. treatment_guidelines.pdf (about 15 pages)
   One section for each of the 10 diagnoses in the transcripts. Each
   section has these sub-headings: Definition, Common symptoms, Tests,
   First-line treatment (drug, dose, frequency, days), Second-line
   treatment, Danger signs, When to refer, Follow-up interval.
   - For 8 diagnoses, the first-line treatment must match the
     transcript's prescription exactly.
   - For 2 diagnoses (pick malaria and one other), make the guideline
     differ: one with a different dose, one where the prescribed drug is
     listed as second-line. Record which two in
     data/knowledge_base/planted_mismatches.json.

2. hospital_procedures.pdf (about 8 pages)
   Sections: Triage levels (red, yellow, green, with examples),
   Referral process, Discharge process, Follow-up booking rules,
   Patient education basics.

3. drug_list.pdf (about 7 pages)
   One entry per drug, including every drug in the transcripts plus 10
   other common outpatient drugs. Each entry: name, form, adult dose,
   child dose, maximum daily dose, NHIS covered (yes/no), NHIS tariff
   in GHS (fictional). Make 3 drugs NOT NHIS covered.

Use plausible, consistent clinical wording, but this is fictional.
After generating, print: page count per PDF, list of section headings,
and the contents of planted_mismatches.json.
```

Prompt B: the 30 test questions

```
Read the three Markdown source files in data/knowledge_base/ and
create evals/module2_questions.json with 30 test questions.

Each item: id (Q01...), question, expected_answer (one or two
sentences), expected_document, expected_section, answerable
(true/false).

Mix:
- 12 questions about treatment and doses (treatment_guidelines)
- 6 about procedures: triage, referral, discharge (hospital_procedures)
- 5 about drugs: max dose, NHIS coverage, tariff (drug_list)
- 2 that need two documents, e.g. "Is the first-line drug for
  typhoid covered by NHIS?"
- 5 that are NOT answerable from the documents (e.g. broken bones,
  surgery, conditions not covered); expected_answer = "This is not
  covered in the clinic guidelines."

Write questions the way clinic staff would ask them: short, sometimes
informal, some using drug brand-style names or abbreviations.
```

When it's done, check two things yourself: open the malaria section of the guidelines and confirm the planted mismatch is really there, and read 5 test questions to make sure their expected answers are actually in the documents.